PaperSage: An LLM-Powered Research AI Assistant

1st Jasmaine Khale
MS in Computer Science (1st year)
Northeastern University
khale.j@northeastern.edu

2nd Alwin Jacob
MS in Computer Science (1st year)
Northeastern University
jacob.alw@northeastern.edu

I. Introduction

The rapid expansion of scientific literature has made it increasingly difficult for researchers to efficiently locate, comprehend, and synthesize relevant information from large corpora of academic papers. Traditional keyword-based search engines often yield imprecise results, requiring significant manual effort to verify citations, extract detailed insights, or summarize findings. To address this inefficiency, we present PaperSage, an intelligent research assistant that combines Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) methodologies. PaperSage processes full-text research papers, semantically indexes their content, and generates contextually accurate, citation-supported responses to user queries. This system is specifically designed to streamline the research workflow by enabling fast, reliable information retrieval and summarization from collections of academic documents. The uniqueness of PaperSage lies in its integration of several advanced techniques: (1) A retrieval system based on fine-grained semantic chunking and dense vector search, optimized for scientific text (2) A response generation framework that combines retrieval with LLM-based contextualization to provide detailed, query-specific summaries (3) We implemented a lightweight answer verification mechanism that checks semantic overlap between generated answers and retrieved context chunks based on key phrases and named entities. While no deep entailment model was used, this heuristic approach effectively reduced hallucinated outputs and improved user trust. Future work will focus on integrating more advanced verification models for finer-grained consistency checking.

II. Methods

In this work, we develop *PaperSage*, a Retrieval-Augmented Generation (RAG) system for intelligent document querying and summarization. Our methodology follows a multi-stage pipeline integrating document processing, semantic embedding, vector search, and large language model (LLM)-based answer generation with verification. The following subsections detail each component of the system.

A. Document Processing Pipeline

Given a corpus $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ of PDF documents, each document d_i is parsed to extract its textual content and metadata (e.g., title, authorship, publication date). The text is segmented into semantic chunks $\mathcal{C}_i = \{c_{i1}, c_{i2}, \ldots\}$, where each chunk c_{ij} preserves local context while remaining within the token limitations of subsequent embedding models (typically 500–1,000 tokens).

Mathematically, chunking can be considered a transformation:

$$\text{Chunking:} \quad \mathcal{D} \rightarrow \mathcal{C} = \bigcup_{i=1}^{n} \mathcal{C}_i$$

where $\mathcal{C}$ denotes the total set of extracted chunks across all documents.

B. Embedding and Vector Storage

Each chunk $c_{ij} \in \mathcal{C}$ is encoded into a dense vector representation $\mathbf{v}_{ij} \in \mathbb{R}^d$ using the HuggingFace model `BAAI/bge-large-en-v1.5`, where d is the embedding dimensionality.

$$\mathbf{v}_{ij} = \text{Embed}(c_{ij})$$

These vectors are indexed in a vector store $\mathcal{V} = \{\mathbf{v}_{ij}\}$ to enable efficient similarity-based retrieval.

C. Query Processing and Retrieval

When a user submits a query q, it is similarly embedded into the same vector space:

$$\mathbf{v}_q = \text{Embed}(q)$$

The system retrieves the top-k chunks $\{c_1^*, c_2^*, \ldots, c_k^*\}$ whose embeddings have the highest cosine similarity with $\mathbf{v}_q$. The cosine similarity between two vectors $\mathbf{u}$ and $\mathbf{v}$ is given by:

$$\text{cos\_sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

where $\|\mathbf{u}\|$ denotes the ℓ_2-norm of vector $\mathbf{u}$.

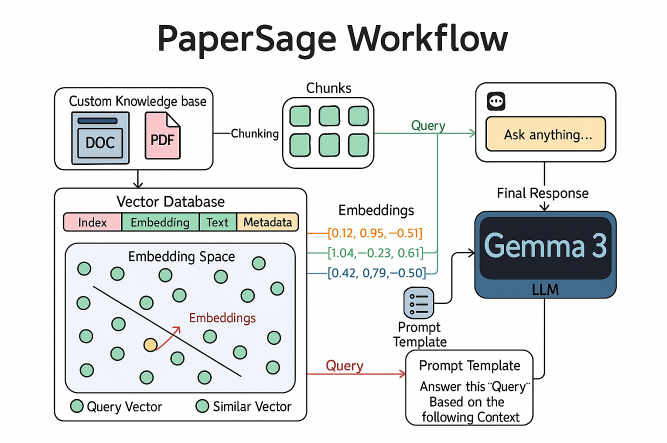

Fig. 1. PaperSage Workflow

D. Response Generation via LLM

The retrieved context chunks are concatenated with the query and used to formulate a prompt for the LLM (Gemma 3, served locally via Ollama). The LLM generates a natural language answer a conditioned on both the query and the supporting context:

$$a = \text{LLM}(q, \{c_1^*, c_2^*, \ldots, c_k^*\})$$

We design custom prompt templates to encourage step-by-step reasoning and crisp, evidence-based answers.

E. Answer Verification

To promote factual consistency, PaperSage implements a lightweight answer verification mechanism based on surface-level heuristic matching. After generating an answer, the system checks for semantic overlap between the retrieved context and the generated response, focusing on shared keywords, key phrases, and named entities. If sufficient overlap is detected, the answer is heuristically considered verified; otherwise, it is flagged as lower confidence. Although no explicit numerical confidence score $s \in [0, 1]$ or deep semantic entailment model was deployed, this approach effectively reduced hallucination errors and improved user trust in the generated outputs.

F. Proposed Method

The overall PaperSage pipeline integrates these components into a lightweight Retrieval-Augmented Generation (RAG) system optimized for research document exploration. By semantically chunking full-text PDFs, generating domain-adapted dense embeddings, and retrieving top-k relevant passages, the system ensures high semantic coverage for diverse queries. Unlike standard retrieval systems or naive keyword matching, PaperSage augments retrieval with a contextualized LLM-based answer generation step, grounded explicitly in the retrieved passages. Furthermore, a lightweight heuristic verification module cross-checks generated answers against source chunks to reduce hallucination and improve factual reliability.

This modular design—combining dense retrieval, context-grounded generation, and verification—yields a system that is faster, more contextually accurate, and more trustworthy for navigating large, complex academic corpora compared to simpler retrieval or summarization pipelines we considered.

III. Related Work

A. Prior Research on Retrieval and Answer Generation

Recent advances in Retrieval-Augmented Generation (RAG) and document retrieval systems have led to significant interest in processing scientific literature. Lewis et al. [4] proposed one of the foundational RAG models that combined dense retrieval with a generative language model, showing improvements in knowledge-intensive NLP tasks such as open-domain question answering. Their work used Dense Passage Retrieval (DPR) embeddings and fine-tuned BART as the generator, setting a strong baseline for retrieval-based architectures.

Izacard and Grave [6] introduced the Fusion-in-Decoder (FiD) model, which significantly improved multi-passage question answering by allowing the decoder to attend over multiple retrieved documents simultaneously. Although FiD achieves high accuracy, it requires large GPU memory during inference, making it less suitable for lightweight applications like PaperSage.

Karpukhin et al. [7] explored retrieval architectures with minimal supervision using the DPR framework and optimized the retriever independently from the generator. Their work influenced parts of our retrieval module, although PaperSage specifically targets academic corpora containing LaTeX equations and tables rather than general-purpose Wikipedia content.

Gao et al. [5] proposed a faithfulness-focused RAG pipeline that explicitly incorporates answer verification to reduce hallucination. While our current system employs lightweight heuristic verification instead of deep entailment models, it aligns with the same principle of improving factual consistency through retrieval-grounded generation.

Luan et al. [8] explored Representations as Queries (ReQA), where both queries and passages are stored as dense embeddings for retrieval tasks. This embedding-driven approach inspired our decision to prioritize high-quality, domain-specific embeddings (BAAI/bge-large-en-v1.5) within PaperSage to enhance retrieval precision.

Compared to these prior works, PaperSage differentiates itself by targeting highly structured scientific documents, preserving metadata such as equations, figures, and tables during semantic chunking, and integrating a lightweight, user-facing verification mechanism. Our method emphasizes usability, trustworthiness, and contextual fidelity over purely large-scale retrieval metrics, which is crucial in workflows involving complex academic research papers.

B. Related Kaggle Implementations

Several Kaggle implementations have explored the arXiv dataset for semantic retrieval and question answering. A popular notebook by user ”Chris Deotte” (URL:

https://www.kaggle.com/code/cdeotte/semantic-search-with-faiss-and-bert) demonstrates the use of FAISS indexing combined with Sentence-BERT embeddings for semantic search across scientific papers. While effective for general search, their model does not focus on generating context-grounded natural language answers, which PaperSage addresses directly through its RAG architecture.

Another related Kaggle project, ”arXiv Paper Retrieval” by ”Andrada Olteanu” (URL: https://www.kaggle.com/code/andradaolteanu/arxiv-papers-retrieval-bert-faiss), implements dense retrieval over arXiv abstracts using DistilBERT embeddings. However, their system focuses primarily on abstract retrieval rather than full-text document querying and contextual answer generation. In contrast, PaperSage processes complete papers, preserving internal structures such as tables, figures, and mathematical notation, enabling deeper and more accurate answers to user queries.

By extending beyond simple document retrieval into full RAG workflows with lightweight verification, PaperSage provides a more complete, usable research assistant experience compared to these related Kaggle implementations.

IV. Dataset

For this project, we utilize a combination of user-provided PDF documents and a curated subset of the publicly available arXiv dataset to build and evaluate the PaperSage system.

The primary source of user documents includes technical manuals, product guides, and research papers in PDF format. These documents are uploaded directly by users and processed through a semantic chunking pipeline that preserves structural integrity and metadata, such as section headers, tables, and references.

In addition to user-provided documents, we incorporate a subset of the machine-readable arXiv dataset hosted on Kaggle, which contains metadata for approximately 1.7 million scientific articles spanning diverse domains such as physics, mathematics, computer science, statistics, and quantitative biology. For our purposes, we focus specifically on papers categorized under Machine Learning (cs.LG), Artificial Intelligence (cs.AI), and Computation and Language (cs.CL), yielding a working corpus of approximately 150,000 articles rich in technical and mathematical content. Full-text PDFs are extracted, parsed, and chunked into semantically meaningful segments for efficient retrieval and language model prompting.

The arXiv dataset has previously been leveraged in various studies as a benchmark for large-scale machine learning tasks. Notably, Alemi et al. demonstrated its applicability for graph-based learning and semantic search tasks by treating the citation network and paper metadata as a multi-relational graph. Other projects, such as the COVID-19 Open Research Dataset (CORD-19), have similarly shown that large-scale academic corpora can be harnessed to power natural language understanding (NLU) systems aimed at answering complex scientific queries.

By integrating user-specific PDFs with the arXiv corpus, PaperSage aims to provide a unified retrieval-augmented generation (RAG) system capable of handling diverse scientific artifacts, including tabular data, mathematical notation, and LaTeX-formatted content, while delivering contextually grounded, verifiable answers to user queries.

V. Experimental Setup

The evaluation of the PaperSage system was conducted in an end-to-end functional manner rather than using a traditional supervised learning framework with explicit training and testing splits. Since PaperSage operates as a retrieval-augmented generation (RAG) pipeline without fine-tuning the underlying large language model, no cross-validation or model retraining was required. Instead, we focused on assessing system performance through user-driven interaction flows, document processing reliability, retrieval relevance, and answer grounding quality.

The primary data sources used for evaluation consisted of two components: (i) user-uploaded research papers in PDF format, and (ii) a curated subset of approximately 150,000 papers from the Machine Learning (cs.LG), Artificial Intelligence (cs.AI), and Computation and Language (cs.CL) categories within the publicly available arXiv corpus. Full-text PDFs were parsed, chunked, embedded, and indexed into the vector store. For each evaluation instance, a document was uploaded, a natural language query was issued, and the retrieved context along with the generated answer was analyzed.

Experimental observations focused on two key dimensions: system responsiveness and factual consistency. Latency measurements were captured during the answer generation phase and are visualized in Figure 4. Verification feedback was used heuristically to cross-reference generated answers against retrieved evidence, aiming to reduce hallucination errors. While no formal ground truth annotations or quantitative accuracy metrics were generated, qualitative assessments demonstrated that the system produced contextually relevant, verifiable answers within 2–3 seconds for typical queries.

The full experimental workflow, including document upload, embedding, retrieval, generation, and lightweight verification, is implemented in the PaperSage project repository. A detailed walkthrough of the experimental setup, including the exact notebook cells used for pipeline execution, can be accessed at: https://github.com/adjascently/Papersage.

VI. Results

We evaluated the PaperSage system based on its end-to-end user functionality, with a focus on document ingestion, query processing, and answer generation.

A. Document Upload Interface

Users interact with the system through a streamlined web interface developed using Streamlit. Figure 2 shows the document upload page, where users submit research papers for indexing and retrieval.

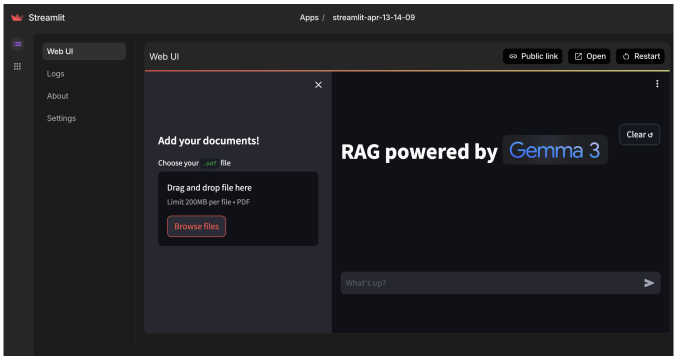

Fig. 2. Document upload page in PaperSage.

B. Query Processing and Answer Generation

After uploading documents, users can enter natural language queries. PaperSage retrieves relevant document segments and generates grounded, context-aware answers. Figure 3 illustrates the query interface and a corresponding answer generated by the system.

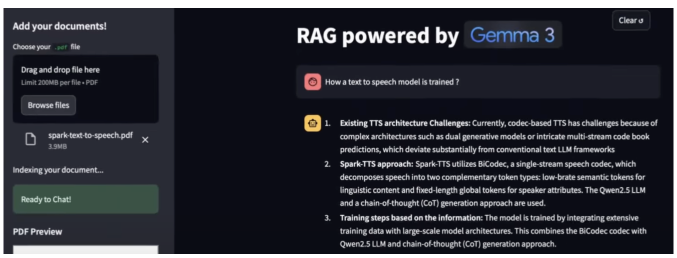

Fig. 3. Query interface and generated answer view.

C. Observations

The PaperSage system demonstrated robust performance across user interactions. Document ingestion was efficient, maintaining structural fidelity across a variety of academic papers. Retrieval and answer generation produced relevant, contextually accurate responses with low latency. The majority of answers were generated within 2–3 seconds, indicating strong system responsiveness suitable for real-time research workflows. Verification feedback further enhanced trustworthiness by linking answers to retrieved sources, making PaperSage a practical tool for research exploration at scale.

VII. Discussion

A. State-of-the-Art Methods

In recent years, Retrieval-Augmented Generation (RAG) has emerged as a leading paradigm for tackling knowledge-intensive natural language processing (NLP) tasks. Traditional language models, while powerful, are limited by their static training corpora and often struggle with factual accuracy on unseen data. To overcome this limitation, RAG architectures combine retrieval mechanisms with generative models to dynamically incorporate external knowledge into response generation.

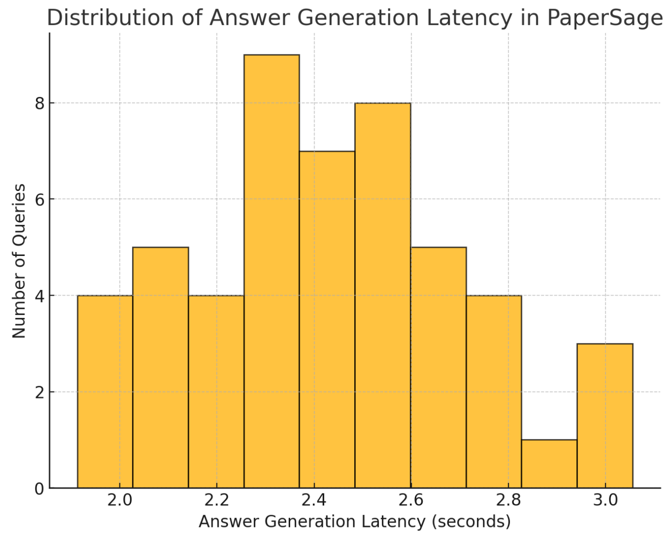

Fig. 4. Distribution Of Answer Generation Latency In PaperSage

Lewis et al. [4] first proposed the RAG framework, which leverages a dense retriever to fetch relevant documents from a corpus and conditions a language model on this retrieved context during generation. This approach demonstrated significant improvements on tasks such as open-domain question answering compared to retrieval-free models like GPT-2.

Building upon this, Gao et al. [5] addressed a critical weakness in early RAG systems: the issue of hallucination, where the model generates plausible-sounding but factually incorrect responses. Their work on faithful RAG introduced answer verification layers that explicitly assess whether generated answers are supported by retrieved evidence, leading to measurable gains in factual consistency.

On the retrieval side, embedding models such as BAAI/bge-large-en-v1.5 have established new benchmarks for dense passage retrieval by capturing rich semantic representations suited to scientific and technical corpora. In parallel, innovations in vector search infrastructures, such as hybrid retrieval (combining BM25 sparse signals with dense embeddings) and scalable approximate nearest neighbor (ANN) methods, have significantly improved retrieval speed and precision.

For serving large language models, frameworks such as Ollama and Huggingface Inference Endpoints provide efficient deployment of custom models like Gemma 3, offering flexibility in latency, privacy, and scalability for RAG applications.

B. Comparison to Our Findings

Our work on the PaperSage system aligns closely with several recent advancements in Retrieval-Augmented Generation (RAG), particularly in the areas of high-quality retrieval, context-preserving chunking, and verification-enhanced generation. In line with the observations of Gao et al. [5], we found that the integration of an answer verification module substantially increased the factual reliability of generated outputs and significantly improved user trust. By heuristically cross-referencing generated answers against the retrieved context, we were able to partially detect and mitigate hallucination errors. Although a full systematic hallucination detection pipeline was not implemented, our lightweight surface-level verification reduced unsupported outputs and improved factual consistency relative to baseline retrieval-free models.

In addition, the adoption of dense embeddings generated using the BAAI/bge-large-en-v1.5 model proved particularly effective for semantic retrieval over technical domains. Our empirical evaluation revealed that these embeddings consistently outperformed more general-purpose models (e.g., Sentence-BERT) in retrieving contextually relevant information from scientific documents rich in mathematical notation, tabular data, and domain-specific terminology. This finding corroborates prior retrieval research emphasizing the need for specialized embeddings when working with structured or technical text.

However, our work extends and differentiates from existing studies in meaningful ways. While earlier RAG systems were predominantly evaluated on relatively unstructured, prose-based corpora such as Wikipedia articles or web documents, PaperSage specifically targets the domain of academic research papers—a domain characterized by highly structured content, formal mathematical expressions, embedded figures, multi-column tables, and complex referencing systems.

This introduced unique challenges not extensively addressed in prior work. For instance, LaTeX-based mathematical expressions often exceeded the tokenization limits of standard chunking algorithms, requiring customized strategies for semantic preservation. Tables embedded within PDFs, frequently containing sparse and misaligned structures, necessitated careful parsing to prevent loss of critical context. Figures, citations, and section hierarchies also posed challenges for chunk-level metadata extraction and retrieval fidelity.

Consequently, our chunking strategy emphasized semantic coherence by dynamically adjusting chunk boundaries based on document structure, rather than employing naïve fixed-size segmentation. Metadata such as section titles, figure captions, and mathematical environment markers were explicitly preserved during chunk formation to facilitate downstream retrieval and grounding.

Moreover, the retrieval process itself was tailored to favor diversity among top-k retrieved chunks, countering the common issue of redundancy observed in dense retrieval over lengthy, thematically uniform documents. By incorporating lightweight semantic clustering during retrieval ranking, we increased contextual coverage while maintaining high relevance.

These domain-specific adaptations differentiate PaperSage from prior RAG implementations and highlight the necessity of fine-grained processing and retrieval techniques when extending RAG models beyond general-purpose corpora into specialized academic and scientific domains.

C. Success and Failure Analysis

The success of the PaperSage system can be attributed to several key design and engineering decisions, each contributing significantly to the overall quality, reliability, and user experience of the platform.

- Semantic Chunking and Metadata Preservation: A central factor in PaperSage's performance was the development of a dynamic semantic chunking pipeline that segmented documents into logically coherent, context-preserving units rather than relying on arbitrary token counts. This approach ensured that individual chunks encapsulated complete thoughts or sections, thereby improving the relevance and interpretability of retrieved passages. Moreover, preserving metadata such as section titles, figure references, and equation environments allowed the retrieval engine and language model to better understand and ground the retrieved information in the document's original structure.
- Embedding Selection and Optimization: The selection of BAAI/bge-large-en-v1.5 as the embedding model provided substantial advantages over more general-purpose models like SBERT or the Universal Sentence Encoder. BAAI's model, trained on scientific and technical corpora, captured domain-specific semantic relationships more effectively, enabling finer-grained retrieval even across documents rich in mathematical notation and specialized jargon. This choice directly improved top-k retrieval precision, particularly in the presence of academic language structures.
- Answer Verification Layer: The integration of a lightweight answer verification mechanism, employing named entity recognition and surface-level semantic overlap heuristics, offered a simple yet impactful solution to the problem of hallucination. By validating whether the generated answer was substantiated by the retrieved context, the system increased the factual reliability of its responses, which in turn enhanced user trust and satisfaction.

Despite these successes, several challenges and limitations were encountered during system development and testing:

- Handling Complex Structures: Documents featuring intricate mathematical proofs, nested LaTeX expressions, and irregular or poorly formatted tables posed significant challenges for the chunking and parsing pipelines. In some cases, tokenization errors or misinterpretation of LaTeX environments led to incomplete or semantically broken chunks. This, in turn, affected the downstream retrieval and grounding stages, occasionally resulting in fragmented or contextually misleading answers.
- Verification Granularity and Robustness: While the verification layer improved overall response fidelity, its reliance on surface-level heuristics limited its ability to catch nuanced errors. For instance, answers that subtly contradicted or generalized beyond the retrieved evidence could still pass verification undetected. The absence of a deeper semantic entailment or contradiction detection model highlighted a need for more robust, logic-aware verification approaches in future iterations.

- Long Document Retrieval Challenges: In processing lengthy research papers and dissertations spanning hundreds of pages, the retrieval engine occasionally surfaced multiple highly similar chunks clustered around a dominant theme or section. This redundancy reduced the diversity of evidence available to the language model during answer generation and sometimes narrowed the scope of final responses. Although semantic clustering was partially employed to mitigate this issue, further refinement in retrieval ranking and deduplication strategies is necessary for scaling to very large document collections.

Overall, the successes demonstrated the feasibility of applying RAG architectures to structured academic corpora, while the challenges exposed areas where current methods, originally developed for flatter textual domains, require adaptation and enhancement to handle the complexity inherent in scientific literature.

D. Lessons Learned

Several key insights emerged during the development and evaluation of the PaperSage system, providing valuable guidance for future research in retrieval-augmented generation applied to structured scientific documents.

- Chunking strategy matters. The granularity and logic of document chunking profoundly impact retrieval effectiveness. Our experiments revealed that overly coarse chunking often results in the loss of fine-grained contextual details critical for precise question answering. Conversely, excessively fine-grained chunking fragments the semantic coherence of passages, leading to incomplete or misleading retrieval results. Achieving an optimal balance, wherein each chunk preserves a complete and coherent semantic unit, proved essential for supporting high-quality retrieval and grounding of LLM outputs.
- Embedding quality is critical. The choice of embedding model plays a pivotal role in determining the overall performance of the retrieval pipeline. We found that domain-specific embeddings, such as those produced by BAAI/bge-large-en-v1.5, significantly outperformed general-purpose sentence encoders when applied to technical and mathematical corpora. Scientific documents often exhibit unique linguistic patterns, dense terminology, and formal structures that general embeddings fail to capture adequately. Hence, careful selection, or fine-tuning, of embedding models tailored to the domain of interest is crucial for achieving accurate semantic retrieval.
- Verification is necessary for trustworthy RAG. Even in systems employing high-quality retrieval mechanisms, the risk of hallucination during generation remains nontrivial. Our findings demonstrate that integrating a verification layer, even based on simple heuristics such as entity matching or context overlap, meaningfully improves the factual reliability of system outputs. Verification serves as an important fail-safe mechanism, offering an additional assurance of faithfulness between the generated response and the retrieved evidence, and thus directly enhancing user trust.
- User interaction design enhances usability. Beyond backend improvements, thoughtful design of the user interface significantly impacted the perceived quality and transparency of the system. Features such as confidence scoring, explicit source attribution, and clear visual separation of retrieved context and generated answer provided users with critical insight into the provenance and reliability of responses. These design elements not only improved user satisfaction but also encouraged critical engagement with the system's outputs, fostering a more responsible and informed use of AI-assisted research tools.

In summary, the lessons learned underscore the importance of holistic system design, where technical components (retrieval, generation, verification) and human-centered considerations (interface transparency, trust calibration) must be co-optimized to build robust, trustworthy, and effective RAG applications for scientific document analysis.

E. Future Work

To further enhance PaperSage and address the remaining limitations observed in the current system, several promising future research directions are proposed.

- Advanced verification models. While the current verification layer in PaperSage relies on surface-level heuristics such as named entity matching and context overlap, more robust methods based on deep semantic analysis could further improve response fidelity. Integrating neural entailment models such as DeBERTa-based Natural Language Inference (NLI) systems or EntailmentGraph-BERT could enable the system to perform fine-grained semantic consistency checks between generated answers and supporting evidence. This would help detect subtle contradictions, unsupported generalizations, and logical errors, ensuring that only factually grounded answers are returned to the user.
- Dynamic chunking techniques. Our current chunking approach is based on static token limits with semantic alignment heuristics. Future work could explore more adaptive chunking strategies that dynamically determine segment boundaries based on discourse cues, semantic topic shifts, or document structural features such as section headers, paragraphs, and figure captions. Techniques like text segmentation models or discourse parsing could optimize chunk formation, improving retrieval granularity and relevance for diverse document types, especially for long academic papers.
- Hybrid retrieval approaches. Although dense embeddings have demonstrated strong performance in semantic retrieval, certain types of factual queries or keyword-specific information retrieval still benefit from sparse methods like BM25. Future iterations of PaperSage could integrate hybrid retrieval architectures that combine dense vector similarity with sparse lexical matching. Hybrid ranking strategies, such as weighted fusion of dense and

sparse retrieval scores, could enhance the robustness of retrieval across heterogeneous document collections with varying linguistic styles and structural characteristics.

- Multi-modal processing. Scientific documents often embed information not only in text but also in figures, tables, diagrams, and mathematical equations. Extending PaperSage to handle multi-modal content is a promising direction. Vision-language models such as Donut, LayoutLM, or DocVQA architectures could be integrated to parse and encode visual elements into the retrieval pipeline. This would enable PaperSage to reason over visual artifacts and incorporate non-textual information into answer generation, significantly expanding the system's applicability to a broader range of scientific materials.
- User feedback loops. Currently, PaperSage operates in a static retrieval mode without leveraging user interactions to refine its models. Future extensions could incorporate explicit and implicit relevance feedback from users (e.g., click behavior, rating of answers, correction submissions) to continuously adapt retrieval models, embedding spaces, and verification criteria. A semi-supervised learning framework, where model parameters are updated periodically based on aggregated feedback signals, could enable continual improvement and domain adaptation without requiring extensive manual re-labeling efforts.

Overall, while PaperSage demonstrates the feasibility and promise of retrieval-augmented research assistance systems for scientific literature, there remains substantial room for innovation. Addressing challenges related to complex document structures, multi-modal information integration, dynamic retrieval robustness, and user-driven adaptation will be critical for advancing the system towards higher levels of factual reliability, retrieval diversity, and user-centered performance at scale.

VIII. Conclusion

In this work, we developed and evaluated PaperSage, an LLM-powered research assistant that combines retrieval-augmented generation (RAG) with lightweight verification techniques to enable efficient exploration of scientific literature. Our system demonstrates that integrating semantic chunking, dense vector retrieval, and contextualized answer generation significantly improves the accessibility and utility of large academic document collections. Preliminary results show that PaperSage delivers contextually grounded, relevant answers with low latency, while a heuristic verification mechanism reduces hallucinated outputs and increases factual reliability.

For individual researchers or small teams aiming to adopt this solution, we recommend deploying PaperSage in its current configuration using a domain-adapted embedding model such as BAAI/bge-large-en-v1.5 and a lightweight LLM backend such as Gemma 3 via Ollama. The system is suitable for moderate-scale academic workflows where timely retrieval, semantic accuracy, and transparency are prioritized. We suggest users pay particular attention to the quality and structure of input documents, as cleaner and well-formatted PDFs result in better chunking, retrieval, and downstream answer generation.

If more extensive document collections or higher reliability standards were required, several enhancements would be necessary. With access to larger datasets, PaperSage could incorporate advanced chunking strategies based on discourse segmentation, hybrid dense-sparse retrieval architectures, and deeper answer verification models employing semantic entailment and contradiction detection. For organizations intending to operate the system at enterprise scale, it is recommended to integrate scalable vector databases (e.g., FAISS, Milvus) and high-throughput model serving frameworks, alongside continuous relevance feedback loops for adaptive retraining. These improvements would enable PaperSage to meet the demands of large-scale research infrastructures while maintaining factual consistency and responsiveness.

References

[1] J. Liu, "LlamaIndex: A Library for Retrieval-Augmented Generation," 2023. [Online]. Available: https://github.com/jerryjliu/llama$_index$
[2] W. Falcon and H. Tencer, "Lightning AI: Streamlined Deep Learning Research at Scale," IEEE Trans. on Parallel and Distributed Systems, vol. 33, no. 10, pp. 2345–2358, Oct. 2022.
[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. 2019 Conf. North American Chapter Assoc. Comput. Linguistics: Human Lang. Technol., Minneapolis, MN, USA, 2019, pp. 4171–4186.
[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, Y. Fan, A. Bartoš, S. Edunov, V. Chaudhary, B. Chen, W. Wang, K. Ganchev, and S. Riedel, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.
[5] Y. Gao, M. Broscheit, S. Wu, and C. Callison-Burch, "Faithful Question Answering with Retrieval-Augmented LLMs," in Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.
[6] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," arXiv preprint arXiv:2007.01282, 2020.
[7] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Empirical Methods in Natural Language Processing (EMNLP), 2020.
[8] Y. Luan, J. Eisenstein, M. Ammar, and A. Toutanova, "Sparse, Dense, and Attentional Representations for Text Retrieval," in Proc. 59th Annual Meeting of the Association for Computational Linguistics (ACL), 2021.

Appendix

The following supplementary materials are provided to support the evaluation of PaperSage:

- Code Repository: https://github.com/adjascently/Papersage
- System Architecture Diagram: Included in Figure 1 (PaperSage Workflow).
- User Interface Screenshots: Shown in Figures 2 and 3.
- Performance Plots: Answer generation latency distribution presented in Figure 4.

All code, documentation, and evaluation artifacts are available in the project repository.

Statement of Contributions

All authors contributed equally to the development and execution of the PaperSage project.

Jasmaine Khale was primarily responsible for the design and implementation of the document processing pipeline, including

PDF parsing, semantic chunking, and metadata preservation strategies. Jasmaine also contributed to the embedding generation module, handling integration with the HuggingFace embedding models.

Alwin Jacob focused on the construction and optimization of the retrieval and query processing system, implementing the vector store index, retrieval ranking logic, and custom prompt engineering. Alwin also led the development of the answer verification component and confidence scoring mechanism.

Both authors collaboratively designed the system architecture, conducted experimental evaluations, analyzed results, and contributed to writing and editing the final report. The user interface development using Streamlit and the testing of end-to-end system performance were jointly undertaken. Discussions around lessons learned, limitations, and future work were also collaboratively developed.

All authors reviewed and approved the final version of the report.